

| 모바일 액션 회피 게임

History

터치 기반 모바일 액션 회피 게임. 보스의 다양한 패턴을 피하며 멀티터치를 통해 라인을 그어 반격하는 게임플레이. 첫 번째 완성 프로젝트로 게임 개발의 전반적인 흐름을 학습.



1인 개발

기획, 개발 전담



Unity 2021.3

LTS 버전



5+ Systems

Advanced Features



3개월

첫 프로젝트



프로젝트 보기



플레이 영상



GitHub

PROJECT OVERVIEW

게임 개요

터치 기반 모바일 액션 회피 게임. 보스의 다양한 패턴을 피하며 멀티터치를 통해 라인을 그어 반격하는 게임플레이. 첫 번째 완성 프로젝트로 게임 개발의 전반적인 흐름을 학습.





핵심 게임플레이

- ✓ 터치 이동 + 라인 드로잉 공격
- ✓ 3개 스테이지 보스 + 무한 모드
- ✓ Spine 2D 애니메이션 연동



개발 환경

- ✓ 엔진: Unity 2021.3 LTS
- ✓ 기간: 첫 프로젝트
- ✓ 플랫폼: Android Mobile



개발 성과

- ✓ 첫 프로젝트로 게임의 완성까지 달성
- ✓ Spine 외부 애니메이션 툴 연동
- ✓ 3개 보스 + 30개 이상 공격 패턴 구현

기술 스택

터치 기반 모바일 액션 회피 게임

게임 엔진



Unity 2021.3
LTS 버전



Spine Runtime
2D 애니메이션

입력 시스템



Touch Input
멀티 터치 처리



LineRenderer
공격 라인 시각화

패턴 시스템



Coroutine
패턴 순차 실행



Prefab
공격 패턴 재사용

데이터 관리



PlayerPrefs
진행 상황 저장



Singleton
매니저 패턴

CORE FEATURES

핵심 기능 구현

5가지 주요 시스템과 기술적 구현 상세

```
public class SoundManager : MonoBehaviour
{
    public static SoundManager Instance { get; private set; }

    [SerializeField] AudioSource bgmSource;
    [SerializeField] AudioSource sfxSource;

    private void Awake()
    {
        if (Instance == null)
        {
            Instance = this;
            DontDestroyOnLoad(gameObject);
        }
        else
        {
            Destroy(gameObject);
        }
    }

    public void PlayBGM(AudioClip clip)
    {
        bgmSource.clip = clip;
        bgmSource.Play();
    }

    public void PlaySFX(AudioClip clip)
    {
        sfxSource.PlayOneShot(clip);
    }
}
```

1. 매니저 시스템

→ Singleton 패턴 적용

- SoundManager: BGM/SFX 전역 관리
- PatternManager: 공격 패턴 프리팹 생성
- ParticleManager: 파티클 이펙트 관리

→ 구현 목적

- 씬 전환 시에도 리소스 유지
- 어디서든 접근 가능한 전역 관리자

```
public class PlayerController : MonoBehaviour
{
    public static int Hp;
    public static bool isDie;
    public static bool atkState;

    void Update()
    {
        if (isDie) return;

        // 첫 번째 터치: 이동
        if (Input.touchCount > 0)
        {
            Touch touch = Input.GetTouch(0);
            Vector3 touchPos = Camera.main.ScreenToWorldPoint(touch.position);
            touchPos.z = 0;
            transform.position = Vector3.Lerp(
                transform.position, touchPos, moveSpeed * Time.deltaTime);
        }

        // 두 번째 터치: 공격 라인
        if (Input.touchCount > 1 && atkState)
        {
            Touch attackTouch = Input.GetTouch(1);
            DrawAttackLine(attackTouch);
        }
    }

    public void DecreaseHp(int damage)
    {
        Hp -= damage;
        if (Hp <= 0) isDie = true;
    }
}
```

2. 터치 입력

→ 멀티 터치 처리

- Touch 1: 플레이어 캐릭터 드래그 이동
- Touch 2: 공격 라인 드로잉 (LineRenderer)

→ 상태 관리

- isDie: 사망 상태 플래그
- atkState: 공격 가능 상태

```
public class Stage1_Boss : MonoBehaviour
{
    [SerializeField] GameObject[] patterns;
    private int currentPhase = 0;

    IEnumerator BossRoutine()
    {
        while (!isDead)
        {
            yield return StartCoroutine(Phase1Pattern());
            yield return StartCoroutine(Phase2Pattern());
            yield return StartCoroutine(Phase3Pattern());
        }
    }

    IEnumerator Phase1Pattern()
    {
        // 패턴 1: 직선 공격
        PatternManager.Instance.SpawnPattern(patterns[0]);
        yield return new WaitForSeconds(2f);

        // 패턴 2: 부채꼴 공격
        PatternManager.Instance.SpawnPattern(patterns[1]);
        yield return new WaitForSeconds(1.5f);

        // 패턴 3: 원형 공격
        for (int i = 0; i < 8; i++)
        {
            float angle = i * 45f;
            SpawnCircularAttack(angle);
        }
        yield return new WaitForSeconds(3f);
    }
}
```

3. 보스 패턴

→ 코루틴 기반 패턴 시스템

- Stage1 사무라이: 9개 패턴 + 5개 오버랩
- Stage2 바이킹: 8개 패턴 + 특수 패턴
- Stage3 나이트: 다수의 복합 패턴

→ 구현 방식

- 각 패턴을 프리팹으로 제작
- PatternManager에서 생성/관리

```
public class Damagable : MonoBehaviour
{
    [SerializeField] int damage = 1;
    [SerializeField] bool destroyOnHit = true;

    private void OnTriggerEnter2D(Collider2D other)
    {
        if (other.CompareTag("Player"))
        {
            PlayerController player = other.GetComponent<PlayerController>();
            if (player != null)
            {
                player.DecreaseHp(damage);

                // 피격 이펙트
                ParticleManager.Instance.PlayHitEffect(
                    transform.position);

                // 피격 사운드
                SoundManager.Instance.PlaySFX(hitSound);

                if (destroyOnHit)
                    Destroy(gameObject);
            }
        }
    }
}
```

4. 데미지 시스템

→ 충돌 판정 구조

- Parent Collider: 보스 본체
- Child Collider: 공격 오브젝트
- Damagable.cs: 충돌 시 데미지 전달

→ 처리 흐름

1. 공격 오브젝트가 플레이어와 충돌
2. Damagable에서 충돌 감지
3. PlayerController.DecreaseHp() 호출


```
using Spine.Unity;

public class SpineAnimController : MonoBehaviour
{
    [SerializeField] SkeletonAnimation skeletonAnim;

    public void PlayAnimation(string animName, bool loop = false)
    {
        skeletonAnim.AnimationState.SetAnimation(0, animName, loop);
    }

    public void PlayAttackAnimation(int patternIndex)
    {
        string animName = $"attack_{patternIndex}";
        PlayAnimation(animName, false);
    }

    public void OnAnimationComplete(string animName)
    {
        if (animName.Contains("attack"))
        {
            PlayAnimation("idle", true);
        }
    }
}
```

5. Spine 애니메이션

→ 외부 툴 연동

- Spine 2D로 제작된 캐릭터 애니메이션
- Unity Spine Runtime 사용
- 보스별 다양한 공격 모션

→ 적용 대상

- 플레이어 캐릭터
- 3종 보스 (사무라이, 바이킹, 나이트)
- 각 보스별 Idle, Attack, Hit, Die 상태

디자인 패턴

확장 가능하고 유지보수하기 쉬운 코드 아키텍처



Singleton

SoundManager, PatternManager,
ParticleManager 전역 관리

✓ 적용 완료



Coroutine

보스 패턴 순차 실행, 타이밍 제어

✓ 적용 완료



Prefab

30+ 공격 패턴 재사용 가능한 프리팹화

✓ 적용 완료



Component

Damagable, SpineAnimController 모듈화

✓ 적용 완료

ACHIEVEMENTS

주요 성과

프로젝트를 통해 달성한 기술적 성과와 학습 내용



기술적 성과

- ★ 첫 프로젝트로 게임의 완성까지 달성
- ★ Spine 외부 애니메이션 툴 연동
- ★ 3개 보스 + 30개 이상 공격 패턴 구현



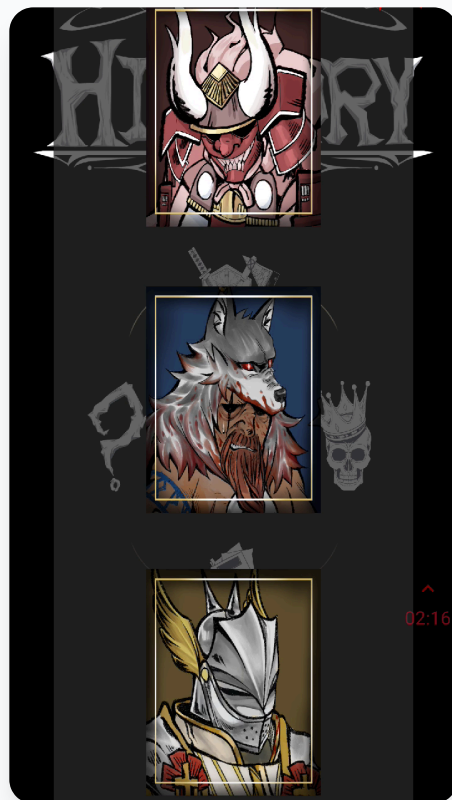
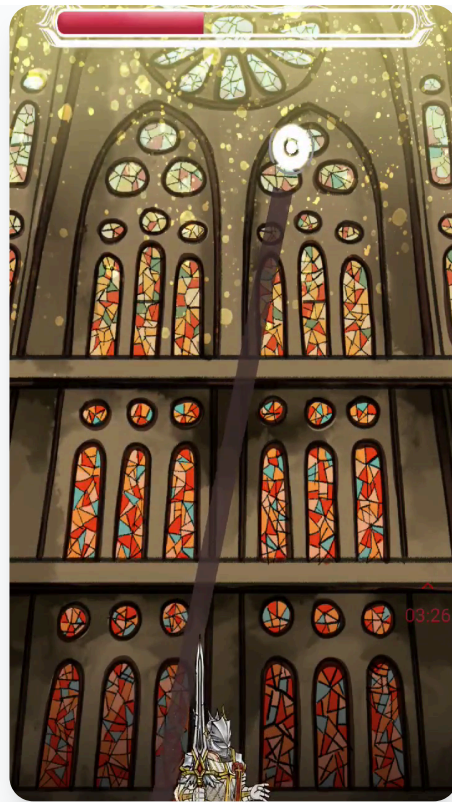
배운점

- ★ 거대한 클래스의 문제점 체감 (3,000줄+)
- ★ Static 변수 과다 사용의 추적 어려움 인식
- ★ 하드코딩 대신 데이터 분리의 필요성 학습
- ★ 상속과 인터페이스 활용의 중요성 깨달음

SCREENSHOTS & VIDEOS

게임 미디어

게임 플레이 스크린샷과 영상



History 플레이영상

junHyeong



다음에서 보기:



History

로 개발한 모바일 액션 회피 게임 포트폴리오

© All Rights Reserved

프로젝트 링크



GitHub Repository



Play Video



홈으로 돌아가기

기술 스택

Unity

C#

2D

Mobile

First Project